

Cahier des Charges : Système NIDS

1. Cahier des Charges :

A. Contexte et Objectif du Projet

- **Projet** : Conception et développement d'un Système de Détection d'Intrusion Réseau Intelligent (NIDS - Network Intrusion Detection System).
- **Objectif** : Utiliser des algorithmes d'Intelligence Artificielle (Machine Learning / Deep Learning) pour analyser le trafic réseau en continu et distinguer les connexions légitimes des attaques cybernétiques (telles que le Déni de Service "DoS" ou le balayage de ports "Port Scanning").

B. Besoins Fonctionnels (Ce que le système doit faire)

- Le système doit permettre à l'administrateur réseau de s'authentifier de manière sécurisée (via JWT et contrôle d'accès basé sur les rôles - RBAC).
- Le système doit être capable de capturer le trafic réseau, que ce soit à partir de fichiers enregistrés (fichiers .pcap) ou en temps réel depuis une interface réseau.
- Le système doit analyser les paquets capturés en utilisant un modèle d'IA préalablement entraîné, fonctionnant comme un microservice d'inférence indépendant.
- Le système doit afficher les résultats sur un tableau de bord (Dashboard) interactif, mis à jour en temps réel (via WebSockets), incluant : l'adresse IP source, le type d'attaque identifiée, et l'heure de l'incident (Timestamp).
- Le système doit générer et afficher des alertes visuelles instantanées lorsqu'une menace est détectée.
- Le système doit inclure une couche d'audit et de journalisation dédiée pour garder une trace des alertes, des décisions du modèle et des actions des administrateurs.

C. Besoins Non-Fonctionnels (Comment le système doit fonctionner)

- **Précision** : Le modèle d'IA doit avoir un taux de détection supérieur à 90 % pour minimiser les faux positifs.
- **Performance** : L'analyse des paquets et le déclenchement des alertes doivent se faire en temps réel (ou avec une latence minime).
- **Environnement de test** : Le système sera déployé et testé dans un environnement virtualisé simulant un réseau réel (à l'aide de VirtualBox ou GNS3).

D. Stack Technique (Révisée et Simplifiée)

1. Cœur de l'Application & Services

- **Service IA / Détection** : Python avec Scikit-Learn ou TensorFlow (structuré comme un microservice d'inférence indépendant).
- **Backend / API** : Node.js avec Express.js (TypeScript).

- Frontend (Dashboard) : React.js avec TypeScript et TailwindCSS.

2. Données & Communication

- Base de données : MongoDB (suffisant pour le MVP ; Elasticsearch optionnel si une recherche avancée de logs devient nécessaire).
- Streaming / Pipeline : Redis Streams pour le traitement de données léger et en temps réel.
- Temps réel : WebSockets / Socket.io pour les alertes instantanées sur le frontend.

3. Réseau & Sécurité

- Moteur NIDS : Zeek ou Suricata (choisir un seul moteur d'analyse pour démarrer).
- Sécurité Auth : JWT + RBAC (Role-Based Access Control) pour l'accès sécurisé des administrateurs.
- Couche d'Audit : Système de logs dédié pour le suivi des alertes et actions critiques.

4. Infrastructure & DevOps

- Déploiement : Docker, Docker Compose, Nginx.
- Monitoring & Observabilité : Prometheus, Grafana.
- Environnement de Test : VirtualBox, Wireshark (pour la simulation et la capture .pcap).

2. Diagramme des Cas d'Utilisation (Use Case Diagram)

Ce diagramme illustre les interactions possibles entre l'utilisateur principal et l'application.

Acteur principal : Administrateur Réseau.

Système : NIDS.

Cas d'utilisation principaux :

- S'authentifier : (Condition préalable obligatoire, via JWT/RBAC, reliée par un <<include>> aux autres actions).
- Entraîner le modèle IA : Charger un Dataset pour l'apprentissage du système.
- Lancer la surveillance du réseau : Démarrer le processus de capture et d'analyse des paquets.
- Consulter le Dashboard : Visualiser les statistiques globales du réseau (trafic normal vs trafic malveillant) alimentées en temps réel via WebSockets.
- Gérer les alertes : Consulter les détails d'une attaque spécifique détectée par l'IA.

3. Diagramme de Séquence (Sequence Diagram)

Ce diagramme montre l'ordre chronologique des messages échangés entre les différents composants du système lors d'un scénario précis.

Scénario : "Détection d'une attaque réseau en temps réel"

Objets/Composants impliqués : 1. Administrateur 2. Interface Web (React Dashboard) 3. Serveur Backend (Node.js API) 4. Modèle IA (Microservice Python)

Séquence des événements :

1. L'Administrateur clique sur le bouton "Démarrer la surveillance" sur l'Interface Web.
2. L'Interface Web envoie une requête (Start Capture) au Serveur Backend.
3. Le Serveur Backend commence à écouter le trafic sur l'interface réseau via le moteur NIDS (Zeek/Suricata).
4. [Un attaquant envoie un paquet malveillant sur le réseau]
5. Le Serveur Backend capture le paquet, en extrait les caractéristiques (features) et les envoie au Modèle IA (Microservice Python).
6. Le Modèle IA analyse les données, identifie une anomalie, et retourne la réponse "Attaque DoS détectée" au Serveur Backend.
7. Le Serveur Backend enregistre l'incident dans la base de données (MongoDB), trace l'événement dans la couche d'audit, et "pousse" une notification d'urgence vers l'Interface Web via WebSockets.
8. L'Interface Web affiche immédiatement une alerte rouge critique à l'Administrateur.